

Lab Exercise 3 - **Wedding Planner**

A Distributed Booking System for
Coordinating Hotel and Band
Reservations

Introduction

In this section, we describe the problem you are expected to solve, which serves as the motivation for developing your **Service-Oriented Architecture (SOA)-based client-server application**. You will be working with a **RESTful Web API** as the backend, though its implementation details are unknown. Your task is to develop the **frontend using Python**, which will interact with the API over the **HTTP protocol** and exchange data in **JSON format**.

To achieve this, you will:

- Retrieve and display data from the API using **HTTP GET requests**.
- Send data to the API (e.g., user input from a UI or command-line tool) using **HTTP POST requests**.
- Process JSON responses and handle errors gracefully.
- Ensure proper communication with the API using Python's built-in **requests** library.

This exercise will help you gain practical experience with **RESTful web services**, API consumption in Python, and structured data exchange using JSON.

Problem Description

You need to arrange a wedding at the earliest available time by making two reservations: one for a **hotel** to host the event and one for a **band** to perform at the reception. Both the hotel and the band advertise availability across **slots 1 to 550**, where **slot 1 is the earliest and slot 550 is the latest**.

Since the band performs at the hotel reception, both reservations must match—if you reserve slot *i* for the hotel, you must also reserve slot *i* for the band. However, this task comes with several challenges:

- **Concurrency:** You are competing with other clients (your classmates), who are also trying to book slots.
- **Service Unavailability:** The hotel and band services may randomly **become unavailable** or introduce **processing delays**.
- **Request Processing Order:** The services **may not process requests in the order they are received**.
- **Network Delays:** When retrieving responses, the requested information may not be immediately available, requiring **retry logic**.
- **Reservation Limits:** Each user can **hold at most two reservations at any time**, meaning that if your client makes a reservation that is no longer needed, it must be **cancelled promptly**.
- **Rate Limiting:** To avoid overwhelming the servers, your client **must not send more than one request per second**.

The services have been configured to randomly make themselves unavailable to incoming requests and to introduce arbitrary delays in processing requests. However, once the server acknowledges the receipt of your message you may assume your message will not be lost.

The hotel and band provide their reservation services via **RESTful web APIs**, which accept **JSON messages over HTTP**. Your client application will need to:

1. **Query the available slots** from both services.
2. **Identify and reserve the earliest matching slot**.
3. **Handle failures gracefully** and **retry reservations** if necessary.
4. **Cancel any unnecessary reservations** to comply with the reservation limit.
5. **Follow the one-request-per-second rule** to maintain fair usage of the services.

This exercise is designed to help you experience the **challenges of distributed systems**, particularly **handling unreliable services, concurrency, and network-related issues** in real-world applications.

Prerequisites

Download Files

You should download the following files from Blackboard:

- **api.ini**: a basic configuration file
- **exceptions.py**: a Python module containing exceptions that are thrown by the ReservationApi.
- **reservationapi.py**: a simple reservation API wrapper.
- **booking.py**: an empty skeleton for your client code.

Get the API keys

Before you can communicate with either the band or hotel APIs, you must obtain an authentication token for each system. These tokens are unique to you, and they are used to identify you in API requests without you having to store or enter a username or password in your client code. To obtain the API keys for Band and Hotel, you should open:

Band: <https://web.cs.manchester.ac.uk/band/>

Hotel: <https://web.cs.manchester.ac.uk/hotel/>

Swagger

Both APIs provide a Swagger interface through which you can try out API calls “by hand” and this is accessible via the “Swagger” link in the menu bar at the top of both the band and hotel API Admin Panels. The Swagger page also includes information about the expected arguments for each API endpoint, and the possible response values you can receive from each endpoint.

To send a reservation request to the band you need to perform a HTTP POST operation message to the URL

Finding free Slots using Swagger

To give some idea of how Swagger works we detail here how to find available free slots.

1. Log into the [API dashboard](#).
2. Copy your API key and navigate to ‘Swagger’.
3. Locate the ‘Authorize’ button, and provide your API key.

4. Locate the GET endpoint '/reservation/available', and click the 'GET' button. This will provide you with information about the endpoint, including a list of responses possible from the endpoint – these will be useful for when you implement the client.
5. Locate the 'Try it out', once you have clicked this button you will be presented with another button 'Execute'.
6. Once you have clicked the 'Execute' button you will receive a response from the server. If all went well, the response code will be 200 and the response body will contain a list of available slots. For example:

```
[{"id": "14"}, {"id": "15"}, {"id": "16"}, {"id": "17"}, {"id": "18"}, {"id": "20"}, {"id": "22"}, {"id": "24"}, {"id": "25"}, ... , {"id": "534"}, {"id": "538"}, {"id": "539"}, {"id": "540"}, {"id": "541"}, {"id": "542"}, {"id": "543"}, {"id": "544"}, {"id": "545"}, {"id": "546"}]
```

If all did not go well, you will receive another response code, for example, 503 Error: Service Unavailable, which would provide the response body:

```
{"code": "503", "message": "Service Unavailable"}
```

API calls from Python

Requests to the APIs must include your API Key in the request header field "Authorization" in the following format:

Authorization: Bearer <API Key>

For example: **Authorization: Bearer f872be7ee40294....87674b**

This header must be created and returned by the '**headers**' method defined within the **reservationapi.py** file.

GitLab and Code Security

It is your responsibility to ensure that your code and API keys remain private and are not accessible to your peers. When setting up your GitLab repository, make sure it is set to **private**. This will help prevent unauthorized access to your code.

Be aware that **collusion**—working with other students to share or copy code—will be considered a violation of academic integrity. If another student submits the same code as yours, both parties could face consequences. This assessment is an individual task, and you must complete it on your own.

You have been warned: protect your work and ensure that you are not sharing your code or collaborating inappropriately. Plagiarism or collusion will lead to serious academic consequences.

Prerequisites Conclusion

Once you have:

- downloaded the necessary files and spent some time trying to understand what the code is doing there
- experimented with Swagger to get a better understanding of the API calls you can make and the possible responses
- configured the **reservationapi.py** file so that the header is created and returned by the calling method/function (and any other configuration files).

You are ready to read the assessment brief and begin your assessment.

Application Requirements

The Wedding Planner Application aims to help a user book a hotel and a band for a wedding reception. The client application will interact with two external APIs—one for the hotel and one for the band. The Booking Orchestrator will act as the intermediary, coordinating the reservations for both the hotel and band, ensuring that both bookings are made for the same slot and managing retries, concurrency issues, and service availability.

- Allows users to:
 - View the current slots held for the hotel and band.
 - View the first 20 available slots for the hotel and band.
 - Allow the booking of an available slot specified by the user.
 - Cancel the booking of a given held slot.
 - View the first 5 matching available slots for the hotel and band.
 - Reserve the earliest matching slot.
 - Cancel any unneeded reservations.
- Ensures that one request per second is sent to avoid overwhelming the APIs.

You should design and implement a frontend for the Booking Orchestrator that facilitates the user actions, credit is given for visually enhancing the frontend if user experience is improved, however, the menu of the frontend can be command line driven; you do not need to implement a graphical user interface unless you wish – credit is not given for a graphical user interface – credit is given for an improved user experience.

While the user should be able to manually select options (e.g., **Cancel the booking of a given held slot**), you should also consider enhancing the user experience by automating certain tasks where appropriate. For example, upon the initial run of the program, it could **automatically display the list of currently held slots**. This provides an immediate overview of the system's current state, improving usability.

Additionally, you may want to automate the deletion of **slots that are booked** when a **matching pair of slots** (hotel and band) is successfully reserved. This reduces the need for the user to manually clean up old or conflicting bookings, streamlining the process.

However, it is essential that **markers** (e.g., tutors or assessors) of the application are also given the ability to perform these automated tasks manually. Even though some tasks are automated for the user's convenience, the marker should still have the option to override or perform these tasks themselves to ensure full control during the review process.

When coding the application you should follow best programming practices. You should optimise where you can, for example, reducing the number of API calls where possible. There should be clear output for each option available, and each option selected. If an option fails, the system should retry until the maximum number of retries is reached. The application should handle errors or retries appropriately. It should be clear if an action is being retried to the user.

Although it is important to secure applications, in this case you can assume that will be handled by another party. You do not need to implement authentication for users of the application or any other

form of security. However, it is quite easy, by having a number of clients fire requests at the server quickly for this to turn into a denial-of-service attack! You are responsible for ensuring that this does not happen. Code which loops around interrogating the server should include a deliberate delay to reduce the danger of this.

Threads and locks are not required. However, if you choose to use them, ensure you include a sufficient time delay between calls to the same API.

To summarise the requirements, you need to have a fully operational program (client) that implements a strategy for booking matching slots that relies on availability, does not give up when a booking request fails, and will cancel unmatched bookings. Furthermore, your strategy should find the earliest matching slot, not any matching slot; and, once a matching slot is found, it should also check for a better booking; you need to make sure that you cancel bookings after obtaining better ones, but you need to maintain always at least one booking; also, your code should not give up if no matching slots are available unless the maximum number of retries is exceeded. Finally, there must be a delay of at least 1 second between successive requests to the server (even if it is a retry).

Submitting and Marking

1. It is important that you test all your lab work on one of the lab machines before submitting it, making sure that your code solution runs in the lab environment, assuming that your code will be tested on one of those machines. Although the machines in the labs of the Kilburn Building are all dual-boot (with Windows and Linux), in this course unit, we will be using Linux.

Note: when using a lab machine use python3 rather than python at the terminal if you have issues with importing libraries such as simplejson.

2. Submit a zipped file using the standard ZIP format compression algorithm (Deflate) to Blackckboard using the naming convention **lab3_username.zip**. The zip file should contain:
 - api.ini
 - exceptions.py
 - reservationapi.py
 - booking.py

Warning: If the GTA cannot download or run your file you will be awarded zero. Do not encrypt your zip file, make sure your api keys are included in the appropriate file and make sure it runs on the version of Linux in the lab. Do not use other compression methods like .tar or .7zip - make sure you have used ZIP to compress otherwise the GTA may not be able to access the files in the zipped folder.

3. Much of the marking is the testing of the application functionality. If the menu options are clear, and the output is unambiguous, and it is clear from the output what the program is doing or has done, you will achieve more marks. The marker is not going to inspect your code for functionality.
4. The marker will check your code for other aspects of the marking scheme, for example, to determine if best programming practices have been followed.

5. Credit is given to encourage robust implementations. The program should not crash, the user must not resort to sending a SIGINT (interrupt) signal to stop a running process, the marker should not have to rerun the program to resume the marking process, etc.